



PostgreSQL

Unnormalized Form

...

S#	City	Status	P#	Qty
S1	Surat	20	P1	1000
			P2	500
			P3	700
			P4	800
			P5	1100
			P6	900
S2	Baroda	10	P1	300
			P2	1200
S3		10	P2	1000
S4	Surat	20	P2	200
			P4	300
			P5	500
S5	Ahemdabad	30	P1	10000

1st Normal Form

S#	City	Status	P#	Qty
S1	Surat	20	P1	1000
S1	Surat	20	P2	500
S1	Surat	20	P3	700
S1	Surat	20	P4	800
S1	Surat	20	P5	1100
S1	Surat	20	P6	900
S2	Baroda	10	P1	300
S2	Baroda	10	P2	1200
S3	Baroda	10	P2	1000
S4	Surat	20	P2	200
S4	Surat	20	P4	300
S4	Surat	20	P5	500
S5	Ahemdabad	30	P1	10000

2nd Normal Form

| (S#, P#) → Qty |
Support(S#, P#, Qty)

S#	P#	Qty
S1	P1	1000
S1	P2	500
S1	P3	700
S1	P4	800
S1	P5	1100
S1	P6	900
S2	P1	300
S2	P2	1200
S3	P2	1000
S4	P2	200
S4	P4	300

| S# → City, City → Status |
Supplier(S#, Status, City)

S#	City	Status
S1	Surat	20
S2	Baroda	10
S3	Baroda	10
S4	Surat	20
S5	Ahmedabad	20

3rd Normal Form

$\{(S\#, P\#) \rightarrow Qty\}$
 $Support(S\#, P\#, Qty)$

$\{S\# \rightarrow City\}$
 $SupCity(S\#, City)$

$\{City \rightarrow Status\}$
 $CityStatus(City, Status)$

S#	P#	Qty
S1	P1	1000
S1	P2	500
S1	P3	700
S1	P4	800
S1	P5	1100
S1	P6	900
S2	P1	300
S2	P2	1200
S3	P2	1000
S4	P2	200
S4	P4	300

S#	City	Status
S1	Surat	20
S2	Baroda	10
S3	Baroda	10
S4	Surat	20
S5	Ahmedabad	20

City	Status
Surat	20
Baroda	10
Ahmedabad	20

SQL Injection

```
SELECT * FROM t_user u WHERE u.email='kp@example.com' AND  
u.password='123';
```

```
//”SELECT * FROM t_user u WHERE u.email='“+ val1 +“' AND  
u.password='” + val2+ “”’;
```

```
SELECT * FROM t_user u WHERE u.email='kp@example.com' AND  
u.password='' or '1'='1';
```

```
PREPARE t_user(varchar,varchar) AS
```

```
    SELECT * FROM t_user u WHERE u.email=$1 AND u.password=$2;
```

```
EXECUTE t_user('kp@example.com', '123');
```

```
EXECUTE t_user('kp@example.com', '' or '1'='1');
```

RANK and Row_Number

RANK :Assigns rank with gaps for duplicate values.

Example:

```
SELECT empid, empname, salary, RANK() OVER(ORDER BY salary DESC) AS rank_no  
FROM employee;
```

```
SELECT empid, empname, salary, RANK() OVER(PARTITION BY city ORDER BY  
salary DESC) AS rank_no FROM employee;
```

ROW_NUMBER :Assigns unique row numbers to rows.

Example:

```
SELECT empid, empname, salary, ROW_NUMBER() OVER(ORDER BY salary DESC) AS  
row_no FROM employee;
```

```
SELECT empid, empname, salary, ROW_NUMBER() OVER(PARTITION BY city ORDER BY  
salary DESC) AS row_no FROM employee;
```

ROW_NUMBER:

- Always unique
- No gaps

RANK:

- Same rank for same salary
- Gaps in ranking

CTE (Common Table Expression)

A CTE (Common Table Expression) is a temporary named result set that you can use inside a SQL query to make complex queries more readable and manageable.

Example:

```
WITH high_salary AS (  
    SELECT empid, empname, salary FROM employee WHERE salary > 70000  
)  
SELECT * FROM high_salary;
```

Find department-wise total salary and Show only the top 2 highest-paid employees per department along with department name.

```
WITH dept_salary AS (  
    SELECT d.deptid, d.deptname, SUM(e.salary) AS total_salary,  
    FROM department d JOIN employee e ON d.deptid = e.deptid GROUP BY  
    d.deptid, d.deptname  
)  
, ranked_employees AS (  
    SELECT  
        e.empid, e.empname, e.salary, e.deptid,  
        RANK() OVER ( PARTITION BY e.deptid ORDER BY e.salary DESC) AS dept_rank  
    FROM employee e  
)
```

CTE (Common Table Expression)

```
SELECT
    ds.deptname, re.empid, re.empname, re.salary,
    re.dept_rank, ds.total_employees,
FROM dept_salary ds
JOIN ranked_employees re ON ds.deptid = re.deptid
WHERE re.dept_rank <= 2
ORDER BY ds.deptname, re.dept_rank;
```

ROLLUP

Used to generate subtotals and grand totals.

Example:

```
SELECT deptid, SUM(salary) AS total_salary  
FROM employee  
GROUP BY ROLLUP(deptid);
```

```
SELECT deptid, city, SUM(salary) AS total_salary  
FROM employee  
GROUP BY ROLLUP(deptid, city);
```

- City-wise totals under each department
- Department subtotals
- Grand total

CUBE

Generates all possible combinations of totals.

Example:

```
SELECT deptid, city, SUM(salary) AS total_salary  
FROM employee GROUP BY CUBE(deptid, city);
```

- Department-wise totals
- City-wise totals
- Department + City totals
- Grand total

GROPIING SETS

Generates custom combinations of totals.

Example:

```
SELECT deptid, city, SUM(salary) AS total_salary
FROM employee
GROUP BY GROUPING SETS (
    (deptid, city),
    (deptid),
    ()
);
```

- City + Department totals
- Department totals
- Grand total
- No city-only totals (unlike CUBE)

LIMIT, OFFSET and FETCH

LIMIT : Restricts number of rows.

Example:

```
SELECT * FROM employee ORDER BY salary DESC LIMIT 10;
```

OFFSET : Skips specified number of rows.

Example:

```
SELECT * FROM employee ORDER BY salary DESC LIMIT 10 OFFSET 20;
```

FETCH : Standard SQL for pagination.

Example:

```
SELECT * FROM employee ORDER BY salary DESC FETCH FIRST 10 ROWS ONLY;
```

Pagination using OFFSET + FETCH.

Example:

```
SELECT * FROM employee ORDER BY salary DESC OFFSET 10 ROWS FETCH NEXT 10 ROWS ONLY;
```

JSON Data

Create Table with JSON column

```
CREATE TABLE sample_data (  
    id SERIAL PRIMARY KEY,  
    data JSONB  
);
```

using JSONB instead of JSON because it is faster and allows indexing & advanced queries.

Insert Sample Data

```
INSERT INTO sample_data (data) VALUES  
('{"name": "Alice", "age": 25, "skills": ["C#", "SQL", "JavaScript"]}'),  
('{"name": "Bob", "age": 30, "skills": ["Python", "Django"]}'),  
('{"name": "Charlie", "age": 28, "skills": ["Java", "Spring", "Postgres"]}');
```

Query JSON Data

Select full JSON

```
SELECT * FROM sample_data;
```

Extract JSON field (-> returns JSON, ->> returns text)

-- Get names

```
SELECT id, data->>'name' AS name FROM sample_data;
```

-- Get ages as integer

```
SELECT id, (data->>'age')::int AS age FROM sample_data;
```

Filter by JSON field

-- People older than 27

```
SELECT id, data->>'name' AS name, data->>'age' AS age FROM sample_data  
WHERE (data->>'age')::int > 27;
```

Search inside JSON array

-- Find people who know Python

```
SELECT id, data->>'name' AS name FROM sample_data WHERE data->'skills' @>  
'["Python"]';
```

Update and Delete JSON Data

-- Add a new field

```
UPDATE sample_data  
SET data = jsonb_set(data, '{city}', '"New York"')  
WHERE data->>'name' = 'Alice';
```

-- Update age

```
UPDATE sample_data  
SET data = jsonb_set(data, '{age}', '26')  
WHERE data->>'name' = 'Alice';
```

```
UPDATE sample_data  
SET data = data - 'city'  
WHERE data->>'name' = 'Alice';
```